

Week2

Problem 1

```
import java.util.Arrays;

public class Solution2_1 {

    public int[] runningSum(int[] nums) {

        for (int i = 1; i < nums.length; i++) ;

            nums[i] += nums[i - 1];

        }

        return nums;

    }

    public static void main(String[] args) {

        Solution solution = new Solution();

        int[] nums1 = {1, 2, 3, 4};

        System.out.println("Input: " + Arrays.toString(nums1));

        int[] result1 = solution.runningSum(nums1);

        System.out.println("Output: " + Arrays.toString(result1)); // Expected: [1, 3, 6, 10]

        System.out.println();

        int[] nums2 = {1, 1, 1, 1, 1};

        System.out.println("Input: " + Arrays.toString(nums2));

        int[] result2 = solution.runningSum(nums2);

        System.out.println("Output: " + Arrays.toString(result2)); // Expected: [1, 2, 3, 4, 5]

        System.out.println();

        int[] nums3 = {3, 1, 2, 10, 1};

        System.out.println("Input: " + Arrays.toString(nums3));

        int[] result3 = solution.runningSum(nums3);

        System.out.println("Output: " + Arrays.toString(result3)); // Expected: [3, 4, 6, 16, 17]

    }

}
```

Output-

Input: [1, 2, 3, 4]

Output: [1, 3, 6, 10]

Input: [1, 1, 1, 1, 1]

Output: [1, 2, 3, 4, 5]

Input: [3, 1, 2, 10, 1]

Output: [3, 4, 6, 16, 17]

Problem 2

```
import java.util.Arrays;

public class Solution2_2 {

    public int[] shuffle(int[] nums, int n) {

        int[] result = new int[2 * n];

        for (int i = 0; i < n; i++) {

            result[2 * i] = nums[i];

            result[2 * i + 1] = nums[i + n];

        }

        return result;

    }

    public static void main(String[] args) {

        Solution solution2_2 = new Solution2_2();

        int[] nums1 = {2, 5, 1, 3, 4, 7};

        int n1 = 3;

        System.out.println("Input: " + Arrays.toString(nums1) + ", n = " + n1);

        int[] result1 = solution.shuffle(nums1, n1);

        System.out.println("Output: " + Arrays.toString(result1)); // Expected: [2, 3, 5, 4, 1, 7]

        System.out.println();

    }

}
```

```

int[] nums2 = {1, 2, 3, 4, 4, 3, 2, 1};

int n2 = 4;

System.out.println("Input: " + Arrays.toString(nums2) + ", n = " + n2);

int[] result2 = solution.shuffle(nums2, n2);

System.out.println("Output: " + Arrays.toString(result2)); // Expected: [1, 4, 2, 3, 3, 2, 4, 1]

System.out.println();

int[] nums3 = {1, 1, 2, 2};

int n3 = 2;

System.out.println("Input: " + Arrays.toString(nums3) + ", n = " + n3);

int[] result3 = solution.shuffle(nums3, n3);

System.out.println("Output: " + Arrays.toString(result3)); // Expected: [1, 2, 1, 2]

}

}

```

Output-

Input: [2, 5, 1, 3, 4, 7], n = 3

Output: [2, 3, 5, 4, 1, 7]

Input: [1, 2, 3, 4, 4, 3, 2, 1], n = 4

Output: [1, 4, 2, 3, 3, 2, 4, 1]

Input: [1, 1, 2, 2], n = 2

Output: [1, 2, 1, 2]

Problem 3

```

import java.util.Arrays;

public class Solution2_1 {

    public int removeElement(int[] nums, int val) {

        int k = 0;

        for (int i = 0; i < nums.length; i++) {

            if (nums[i] != val) {

```

```

        nums[k] = nums[i];
        k++;
    }
}

return k;
}

public static void main(String[] args) {
    Solution2_1 solution = new Solution2_1();

    // Test Case 1
    int[] nums1 = {3, 2, 2, 3};
    int val1 = 3;

    System.out.println("Input: nums = " + Arrays.toString(nums1) + ", val = " + val1);
    int k1 = solution.removeElement(nums1, val1);
    System.out.println("Output: k = " + k1);
    System.out.print("Modified Array (first k elements): [");
    for (int i = 0; i < k1; i++) {
        System.out.print(nums1[i] + (i < k1 - 1 ? ", " : ""));
    }
    System.out.println("]");
    System.out.println();

    // Test Case 2
    int[] nums2 = {0, 1, 2, 2, 3, 0, 4, 2};
    int val2 = 2;

    System.out.println("Input: nums = " + Arrays.toString(nums2) + ", val = " + val2);
    int k2 = solution.removeElement(nums2, val2);
    System.out.println("Output: k = " + k2);
    System.out.print("Modified Array (first k elements): [");
    for (int i = 0; i < k2; i++) {
        System.out.print(nums2[i] + (i < k2 - 1 ? ", " : ""));
    }

```

```

    }

    System.out.println("");
}
}

```

Output-

Input: nums = [3, 2, 2, 3], val = 3

Output: k = 2

Modified Array (first k elements): [2, 2]

Input: nums = [0, 1, 2, 2, 3, 0, 4, 2], val = 2

Output: k = 5

Modified Array (first k elements): [0, 1, 3, 0, 4]

Problem 4

```

import java.util.Arrays;

public class Solution2_4 {

    public int removeDuplicates(int[] nums) {

        if (nums.length == 0) {

            return 0;

        }

        int k = 0;

        for (int i = 1; i < nums.length; i++) {

            if (nums[i] != nums[k]) {

                k++;

                nums[k] = nums[i];

            }

        }

        return k + 1;

    }

}

```

```

public static void main(String[] args) {
    Solution2_4 solution = new Solution2_4();

    int[] nums1 = {1, 1, 2};
    System.out.println("Input: nums = " + Arrays.toString(nums1));
    int k1 = solution.removeDuplicates(nums1);
    System.out.println("Output: k = " + k1);
    System.out.print("Modified Array (first k elements): [");
    for (int i = 0; i < k1; i++) {
        System.out.print(nums1[i] + (i < k1 - 1 ? ", " : ""));
    }
    System.out.println("]");
    System.out.println();

    int[] nums2 = {0, 0, 1, 1, 1, 2, 2, 3, 3, 4};
    System.out.println("Input: nums = " + Arrays.toString(nums2));
    int k2 = solution.removeDuplicates(nums2);
    System.out.println("Output: k = " + k2);
    System.out.print("Modified Array (first k elements): [");
    for (int i = 0; i < k2; i++) {
        System.out.print(nums2[i] + (i < k2 - 1 ? ", " : ""));
    }
    System.out.println("]");
}
}

```

Output-

Input: nums = [1, 1, 2]

Output: k = 2

Modified Array (first k elements): [1, 2]

Input: nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]

Output: k = 5

Modified Array (first k elements): [0, 1, 2, 3, 4]

Problem 5

```
import java.util.Arrays;

public class Solution2_5 {

    public int maxSubArray(int[] nums) {
        int maxSoFar = nums[0];
        int currentMax = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currentMax = Math.max(nums[i], currentMax + nums[i]);
            maxSoFar = Math.max(maxSoFar, currentMax);
        }
        return maxSoFar;
    }

    public static void main(String[] args) {
        Solution2_5 solution = new Solution2_5();

        int[] nums1 = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
        int result1 = solution.maxSubArray(nums1);
        System.out.println("Input: " + Arrays.toString(nums1));
        System.out.println("Output (Expected: 6): " + result1);
        System.out.println();

        int[] nums2 = {1};
        int result2 = solution.maxSubArray(nums2);
```

```
System.out.println("Input: " + Arrays.toString(nums2));  
System.out.println("Output (Expected: 1): " + result2);  
System.out.println();
```

```
int[] nums3 = {5, 4, -1, 7, 8};  
int result3 = solution.maxSubArray(nums3);  
System.out.println("Input: " + Arrays.toString(nums3));  
System.out.println("Output (Expected: 23): " + result3);  
System.out.println();
```

```
int[] nums4 = {-5, -1, -3, -4};  
int result4 = solution.maxSubArray(nums4);  
System.out.println("Input: " + Arrays.toString(nums4));  
System.out.println("Output (Expected: -1): " + result4);
```

```
}
```

```
}
```

Output-

Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Output (Expected: 6): 6

Input: [1]

Output (Expected: 1): 1

Input: [5, 4, -1, 7, 8]

Output (Expected: 23): 23

Input: [-5, -1, -3, -4]

Output (Expected: -1): -1

Problem 6

```
import java.util.Arrays;

public class Solution2_6 {

    public int largestAltitude(int[] gain) {
        int currentAltitude = 0;
        int maxAltitude = 0;
        for (int i = 0; i < gain.length; i++) {
            currentAltitude += gain[i];
            maxAltitude = Math.max(maxAltitude, currentAltitude);
        }
        return maxAltitude;
    }

    public static void main(String[] args) {
        Solution2_6 solution = new Solution2_6();

        int[] gain1 = {-5, 1, 5, 0, -7};
        System.out.println("Input: gain = " + Arrays.toString(gain1));
        int result1 = solution.largestAltitude(gain1);
        System.out.println("Output: " + result1); // Expected: 1
        System.out.println();

        int[] gain2 = {-4, -3, -2, -1, 4, 3, 2};
        System.out.println("Input: gain = " + Arrays.toString(gain2));
        int result2 = solution.largestAltitude(gain2);
        System.out.println("Output: " + result2); // Expected: 0
    }
}
```

Output

Input: gain = [-5, 1, 5, 0, -7]

Output: 1

Input: gain = [-4, -3, -2, -1, 4, 3, 2]

Output: 0

Problem 7

```
import java.util.*;

class Solution {

    public List<List<String>> groupAnagrams(String[] strs) {

        HashMap<String, List<String>> map = new HashMap<>();

        for (String str : strs) {

            // Convert string to character array

            char[] chars = str.toCharArray();

            // Sort characters

            Arrays.sort(chars);

            // Convert back to string

            String key = new String(chars);

            // Add string to its anagram group

            if (!map.containsKey(key)) {

                map.put(key, new ArrayList<>());

            }

            map.get(key).add(str);

        }

        return new ArrayList<>(map.values());

    }

}
```

Output-

Input1

strs =

["eat","tea","tan","ate","nat","bat"]

Output

```
[["eat","tea","ate"],["bat"],["tan","nat"]]
```

Expected

```
[["bat"],["nat","tan"],["ate","eat","tea"]]
```

Input2

strs =

```
[""]
```

Output

```
[[""]]
```

Expected

```
[[""]]
```

Input3

strs =

```
["a"]
```

Output

```
[["a"]]
```

Expected

```
[["a"]]
```

Problem 8

```
import java.util.*;
```

```
class Solution {
```

```
    public int[] topKFrequent(int[] nums, int k) {
```

```
        // Count frequency of each number
```

```
        HashMap<Integer, Integer> map = new HashMap<>();
```

```
        for (int num : nums) {
```

```
            map.put(num, map.getOrDefault(num, 0) + 1);
```

```
        }
```

```
        // Bucket array
```

```
        List<Integer>[] bucket = new List[nums.length + 1];
```

```

for (int num : map.keySet()) {
    int frequency = map.get(num);
    if (bucket[frequency] == null) {
        bucket[frequency] = new ArrayList<>();
    }
    bucket[frequency].add(num);
}

// Store answer
int[] result = new int[k];
int index = 0;

// Start from highest frequency
for (int i = bucket.length - 1; i >= 0 && index < k; i--) {
    if (bucket[i] != null) {
        for (int num : bucket[i]) {
            result[index] = num;
            index++;
            if (index == k) {
                break;
            }
        }
    }
}

return result;
}
}

```

Output

Input 1

nums =

[1,1,1,2,2,3]

k =

2

Output

[1,2]

Expected

[1,2]

Input 2

nums =

[1]

k =

1

Output

[1]

Expected

[1]

Input 3

nums =

[1,2,1,2,1,2,3,1,3,2]

k =

2

Output

[1,2]

Expected

[1,2]